

CSS (Cascading Style Sheets)

What is CSS?

CSS (Cascading Style Sheets) is the language used to control the appearance and layout of HTML documents. While HTML defines the structure of a page, CSS defines how it looks colours, fonts, spacing, and layout.

The word "cascading" refers to the order rules are applied. More specific selectors override less specific ones (e.g. an ID selector overrides a class selector, which overrides an element selector).

CSS Syntax

A CSS rule consists of a selector and a declaration block containing property-value pairs:

```
/* selector { property: value; } */

h1 {
  color: blue;
  font-size: 2rem;
}

.card {
  background-color: #ffffff;
  padding: 1rem;
  border: 1px solid #cccccc;
  border-radius: 6px;
}

#main-title {
  font-weight: bold;
  text-align: center;
}

/* Pseudo-class: change style on hover */
a:hover {
  color: red;
  text-decoration: underline;
}
```

The three main selector types:

- **Element selector** (e.g. `h1`, `p`) : targets all elements of that type.
- **Class selector** (e.g. `.card`) : targets any element with that class.
- **ID selector** (e.g. `#main-title`) : targets one unique element per page.

The CSS Box Model

Every HTML element is treated as a rectangular box with four layers. Understanding this is fundamental to controlling spacing and layout:

- **Content** : the actual text, image, or child elements inside.
 - **Padding** : transparent space between the content and the border.
 - **Border** : a line drawn around the padding and content.
-

- **Margin** : transparent space outside the border, separating elements.

```
.box {  
  margin: 20px;      /* Space outside the element */  
  border: 2px solid #333; /* Visible border      */  
  padding: 15px;     /* Space inside the element */  
  width: 200px;      /* Content width          */  
}
```

Flexbox Layout

Flexbox is a CSS layout module that makes it easy to align and distribute items in a row or column without using floats or manual positioning.

```
.container {  
  display: flex;  
  justify-content: space-between; /* Horizontal alignment */  
  align-items: center;           /* Vertical alignment  */  
  gap: 1rem;                     /* Space between items */  
}  
  
.item {  
  flex: 1;                       /* Each item takes equal space */  
  background: #7C3AED;  
  color: white;  
  padding: 1rem;  
  text-align: center;  
  border-radius: 4px;  
}
```

Responsive Design — Media Queries

Media queries apply different CSS rules depending on screen size. This allows the same HTML to look good on mobile, tablet, and desktop.

```
/* Default — mobile first */  
.container {  
  width: 100%;  
  padding: 0 1rem;  
}  
  
/* Tablet and above */  
@media (min-width: 768px) {  
  .container {  
    width: 750px;  
    margin: 0 auto;  
  }  
}  
  
/* Desktop */  
@media (min-width: 1024px) {  
  .container {  
    width: 960px;  
  }  
}
```

CSS Variables (Custom Properties)

CSS variables let you define reusable values in one place. If the colour scheme needs to change, you only update it in the `:root` block rather than throughout the entire stylesheet.

```
:root {
  --primary-color: #7C3AED;
  --font-size-base: 16px;
  --border-radius: 6px;
  --spacing-md: 1rem;
}

.button {
  background-color: var(--primary-color);
  font-size: var(--font-size-base);
  border-radius: var(--border-radius);
  padding: var(--spacing-md);
}
```

Transitions & Animations

CSS transitions create smooth changes between two states (e.g. a button colour on hover). Keyframe animations allow multi-step sequences.

```
/* Smooth hover transition */
.btn {
  background: #7C3AED;
  color: white;
  transition: background 0.3s ease, transform 0.2s ease;
}
.btn:hover {
  background: #5B21B6;
  transform: scale(1.05);
}

/* Keyframe animation */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

.page-header {
  animation: fadeIn 0.6s ease forwards;
}
```